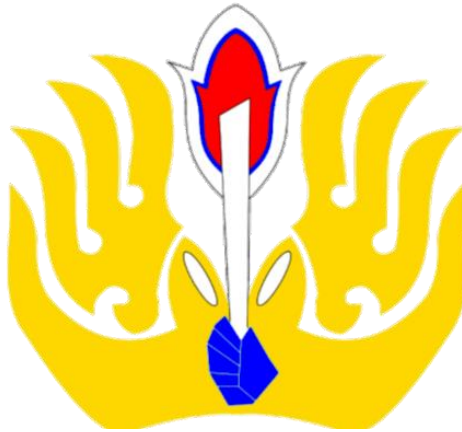


TUGAS LAPORAN
SISTEM MANAJEMEN BIOSKOP



Disusun oleh Kelompok 4 Kelas 2B Sistem Informasi :

Belva Dhia Haqqina	2510631250029
Muhammad Al Fatih	2510631250038
Muhammad Fatih Labib	2510631250071

PRODI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG

BAB I

PENDAHULUAN

Industri hiburan bioskop saat ini menuntut efisiensi yang tinggi dalam pengelolaan operasional harian, terutama pada sistem pelayanan kasir dan administrasi data penayangan. Proses pencatatan transaksi tiket, pengaturan jadwal film, hingga pemantauan denah kursi studio yang dilakukan secara konvensional sering kali menimbulkan risiko kesalahan manusia, ketidakakuratan laporan keuangan, serta lambatnya pelayanan kepada pelanggan. Untuk mengatasi tantangan tersebut, pengembangan aplikasi Sistem Pemesanan Tiket Bioskop berbasis Java ini dirancang sebagai solusi digital terintegrasi. Aplikasi berbasis konsol ini dirancang untuk mengotomatisasi seluruh alur kerja kasir, mulai dari pengelolaan berkas informasi film secara dinamis, penyimpanan data transaksi terstruktur dalam format file penyimpanan terpisah, hingga penyajian laporan statistik penjualan secara seketika guna mempercepat proses pengambilan keputusan dan meningkatkan kualitas layanan operasional bioskop.

Tujuan utama dari perancangan dan implementasi aplikasi ini adalah untuk menyediakan sebuah platform digital penunjang keputusan bagi kasir dan admin yang mampu mengintegrasikan pengelolaan data operasional bioskop secara mandiri. Secara khusus, sistem ini bertujuan untuk memfasilitasi proses penambahan dan pembaruan jadwal tayang film secara teratur, meminimalkan potensi tumpang tindih pemilihan nomor kursi studio melalui sistem denah digital, serta mengamankan riwayat seluruh data transaksi penjualan tiket ke dalam sistem basis data berbasis teks. Melalui pembuatan program ini, diharapkan pihak manajemen bioskop dapat memperoleh laporan ringkasan pendapatan dan statistik film terlaris secara cepat, akurat, dan transparan untuk mendukung efisiensi manajemen bisnis jangka panjang.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Struktur Data

Aplikasi Sistem Pemesanan Tiket Bioskop ini dibangun dengan menerapkan prinsip Pemrograman Berorientasi Objek (*Object-Oriented Programming*) memanfaatkan *Java Collections Framework* serta sistem penyimpanan berbasis berkas teks. Komponen struktur data yang digunakan dalam program ini dibagi menjadi tiga bagian utama, yaitu struktur objek, penyimpanan memori sementara, dan penyimpanan permanen.

2.1.1 Struktur Kelas dan Objek (Data Model)

Sistem ini memecah masalah menjadi tiga kelas model data yang saling berinteraksi secara terstruktur, yaitu:

1. **Kelas Film:** Bertindak sebagai master data yang menyimpan atribut identitas film seperti kode film, judul, genre, nomor studio, durasi penayangan dalam satuan menit, harga tiket, status aktif (untuk kebutuhan akomodasi data riwayat), serta total akumulasi tiket yang telah terjual. Kelas ini juga mengagregasikan data jadwal tayang melalui struktur list objek.
2. **Kelas JadwalTayang:** Menyimpan informasi waktu spesifik penayangan film dalam format jam (HH:mm). Fitur utama dari kelas ini adalah representasi denah studio menggunakan matriks dua dimensi (array 2D) bertipe boolean dengan dimensi 5 baris (mewakili baris A sampai E) dan 10 kolom (mewakili nomor kursi 1 sampai 10). Nilai *true* merepresentasikan kursi yang telah terisi atau dipesan, sedangkan nilai *false* merepresentasikan kursi yang masih kosong.
3. **Kelas Transaksi:** Berfungsi sebagai objek pencatat riwayat (*snapshot memory*) ketika proses pembayaran tiket telah berhasil. Atribut di dalamnya menyimpan salinan data film, waktu transaksi, total bayar, nominal uang tunai yang diterima, uang kembalian, serta daftar nomor kursi yang dibeli dalam bentuk list berbasis teks string.

2.1.2 Struktur Penyimpanan Sementara (In-Memory Storage)

Selama aplikasi dijalankan, data dikelola secara dinamis di dalam memori komputer menggunakan koleksi sebagai berikut:

- **ArrayList:** Digunakan untuk variabel `daftarFilm` dan `daftarTransaksi`. Pemilihan `ArrayList` didasarkan pada karakteristik data yang dinamis (bisa bertambah kapan saja) serta efisiensi performa tinggi saat melakukan akses data berdasarkan indeks, terutama ketika menampilkan data dalam bentuk tabel ke layar pengguna.
- **LinkedHashMap:** Digunakan secara khusus pada fitur *filter* genre film. Struktur ini dipilih untuk menghitung jumlah kemunculan film per genre secara efisien sekaligus menjaga urutan kemunculan data (*insertion order*) agar antarmuka menu pemicu terlihat konsisten dan rapi.

2.1.3 Struktur Penyimpanan Permanen (File-Based Storage)

Untuk menjamin data tidak hilang saat aplikasi ditutup, sistem menerapkan penyimpanan berbasis berkas teks dengan format CSV (Comma-Separated Values). Data disimpan ke dalam dua file terpisah, yaitu `films.csv` dan `transactions.csv`, dengan menggunakan karakter titik koma (;) sebagai pembatas (*delimiter*) antar-kolom atribut. Khusus untuk matriks kursi pada kelas `JadwalTayang`, data diubah menjadi deretan teks biner (karakter '1' untuk terisi dan '0' untuk kosong) agar dapat disimpan dalam satu baris teks CSV secara efisien melalui proses serialisasi teks manual.

2.2 Analisis Fitur Sistem

Sistem Pemesanan Tiket Bioskop ini mengimplementasikan beberapa fitur fungsional utama yang dirancang untuk kebutuhan operasional kasir maupun pengelolaan data oleh administrator.

2.2.1 Manajemen Data Master Film dan Jadwal (CRUD dengan Soft Delete)

Sistem menyediakan antarmuka bagi admin untuk menambah film baru, memperbarui informasi film, serta mengelola jadwal tayang secara dinamis. Fitur penghapusan film menerapkan metode *Soft Delete*, yaitu tidak menghapus baris data secara permanen dari file

database, melainkan mengubah status atribut `aktif` menjadi *false*. Logika ini diterapkan agar transaksi-transaksi lama yang merujuk pada kode film tersebut tidak mengalami kerusakan relasi data (*broken reference*). Sementara itu, fitur modifikasi dan penghapusan jadwal tayang dilengkapi fungsi validasi seketika, di mana jadwal hanya boleh diubah atau dihapus jika belum ada satu pun kursi yang dipesan oleh pelanggan pada jadwal tersebut.

2.2.2 Alur Transaksi Pemesanan Tiket Multi-Tahap (Multi-step Wizard)

Fitur inti dari aplikasi ini adalah proses pemesanan tiket baru yang dirancang dengan panduan bertahap untuk meminimalkan kesalahan input oleh kasir. Alur tahapan transaksi terdiri dari:

1. Input tanggal penayangan dengan validasi format ketat (dd/MM/yyyy).
2. Pemilihan film aktif berdasarkan kode unik.
3. Pemilihan jam tayang yang tersedia pada film tersebut.
4. Input jumlah tiket dengan pengecekan sisa kuota kursi studio yang tersedia.
5. Pemilihan nomor kursi secara interaktif berdasarkan visualisasi denah studio (A1 hingga E10).
6. Penampilan ringkasan nota dan kalkulasi pembayaran (termasuk validasi nominal uang kembalian).

Pada setiap tahapan, sistem menyediakan opsi perintah B untuk kembali ke tahapan sebelumnya (*back*) dan opsi C untuk membatalkan seluruh rangkaian transaksi secara instan (*cancel*) demi fleksibilitas operasional kasir.

2.2.3 Pencarian dan Pengurutan Data Manual (Searching & Sorting)

Untuk menyajikan data secara adaptif, sistem dilengkapi dengan fungsi pencarian kata kunci (*keyword searching*) berbasis teks terisolasi yang mengabaikan huruf besar/kecil (*case-insensitive*) untuk mencocokkan kode, judul, atau genre film.

Selain pencarian, sistem menerapkan algoritma Pengurutan Manual (menggunakan metode *Bubble Sort* atau *Selection Sort*) untuk menyusun daftar film aktif berdasarkan tiga kriteria, yaitu urutan kode film, urutan alfabetis judul film, serta urutan performa penjualan tiket terbanyak (film terlaris). Implementasi pengurutan data secara manual ini menunjukkan optimasi logika pemrograman dasar tanpa bergantung pada fungsi bawaan (*library*) eksternal.

2.2.4 Penyelesaian Status Data (Data Reconciliation)

Saat aplikasi pertama kali dijalankan, sistem secara otomatis mengeksekusi fungsi `reconcileSeatsFromTransactions()`. Fungsi ini bertugas membaca ulang seluruh riwayat transaksi yang ada pada file `transactions.csv`, kemudian secara otomatis mengunci kembali kursi-kursi pada matriks `JadwalTayang` di memori utama. Fitur sinkronisasi otomatis ini menjamin bahwa status keterisian denah kursi studio selalu akurat, valid, dan sejalan dengan transaksi riil yang sudah dibayar oleh pelanggan, meskipun file master film sempat mengalami pembaruan eksternal.

2.2.5 Laporan Pendapatan dan Statistik Penjualan

Sistem secara otomatis mengagregasikan seluruh data penjualan dari file transaksi untuk menghasilkan laporan eksekutif ringkas secara *real-time*. Laporan yang disajikan mencakup total akumulasi transaksi yang terjadi, total kuantitas tiket yang terjual, total omzet pendapatan kotor dalam format mata uang Rupiah, serta penentuan otomatis film terlaris berdasarkan volume penjualan tiket tertinggi.

BAB III

IMPLEMENTASI

3.1 Gambaran Umum Implementasi

Bab ini menyajikan implementasi lengkap kode program Sistem Pemesanan Tiket Bioskop yang dikembangkan menggunakan bahasa pemrograman Java (JDK 17+). Seluruh program dikemas dalam satu file bernama SistemBioskop.java dengan total 1.645 baris kode. Program berupa aplikasi konsol (CLI) dengan dukungan pewarnaan ANSI terminal dan terdiri dari empat lapisan: model data (inner class), penyimpanan berbasis file CSV, algoritma pencarian & pengurutan manual, serta antarmuka pengguna konsol interaktif.

Aspek	Detail
Nama File	SistemBioskop.java
Total Baris Kode	1.645 baris
Bahasa & Versi	Java 17+ (menggunakan switch expression ->)
File Data	films.csv dan transactions.csv
Jumlah Inner Class	3 kelas: Film, JadwalTayang, Transaksi
Algoritma Sorting	Selection Sort manual (3 mode)
Algoritma Searching	Linear Search manual + Sequential Search
Fitur Bonus	Persistensi CSV, ANSI color, Data Reconciliation

3.2 Kode Lengkap Program

Berikut adalah seluruh kode program SistemBioskop.java yang dibagi per seksi logika program beserta penjelasan tiap bagiannya.

3.2.1 Import Library dan Deklarasi Konstanta

Program menggunakan library standar Java (java.io, java.time, java.util) tanpa library eksternal. Seluruh konstanta global dideklarasikan sebagai private static final di bagian atas kelas utama.

SistemBioskop.Java - Bagian 1: Import & Konstanta Global

```
1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.File;
4  import java.io.FileReader;
5  import java.io.FileWriter;
6  import java.io.IOException;
7  import java.io.PrintWriter;
8  import java.time.LocalDate;
9  import java.time.LocalDateTime;
10 import java.time.format.DateTimeFormatter;
11 import java.time.format.DateTimeParseException;
12 import java.time.format.ResolverStyle;
13 import java.util.ArrayList;
14 import java.util.Arrays;
15 import java.util.HashMap;
16 import java.util.LinkedHashMap;
17 import java.util.List;
18 import java.util.Locale;
19 import java.util.Map;
20 import java.util.Scanner;
21
22 public class SistemBioskop {
23     private static final Scanner input = new Scanner(System.in);
24     private static final String FILM_FILE = "films.csv";
25     private static final String TRAN_FILE = "transactions.csv";
26     private static final String APP_TITLE = "SISTEM PEMESANAN TIKET BIOSKOP";
27     private static final int UI_WIDTH = 86;
28
29     private static final boolean USE_COLOR = !"1".equals(System.getenv("NO_COLOR"));
30     private static final String RESET = "\u001B[0m";
31     private static final String RED = "\u001B[31m";
32     private static final String GREEN = "\u001B[32m";
33     private static final String YELLOW = "\u001B[33m";
34     private static final String BLUE = "\u001B[34m";
```

```

35     private static final String MAGENTA = "\u001B[35m";
36     private static final String CYAN = "\u001B[36m";
37     private static final String BOLD = "\u001B[1m";
38
39     private static final DateTimeFormatter DATE_FMT = DateTimeFormatter
40         .ofPattern("dd/MM/yyyy")
41         .withResolverStyle(ResolverStyle.STRICT);
42     private static final DateTimeFormatter TIME_FMT = DateTimeFormatter
43         .ofPattern("HH:mm")
44         .withResolverStyle(ResolverStyle.STRICT);
45
46     private static final int ROWS = 5;
47     private static final int COLS = 10;
48     private static final String CANCEL = "C";
49     private static final String BACK = "B";
50
51     private static final List<Film> daftarFilm = new ArrayList<>();
52     private static final List<Transaksi> daftarTransaksi = new ArrayList<>();
53

```

3.2.2 Metode main() dan Menu Utama

Metode main() adalah entry point program. Saat dijalankan, program memuat data dari CSV, membuat data awal jika file belum ada, menjalankan rekonsiliasi kursi, lalu masuk ke loop menu utama tak hingga hingga pengguna memilih keluar.

SistemBioskop.Java - Bagian 2: Main() dan loop menu utama

```

54     public static void main(String[] args) {
55         loadData();
56         if (daftarFilm.isEmpty()) {
57             seedDataAwal();
58             rebuildTicketStats();
59             saveData();
60         }
61         reconcileSeatsFromTransactions();
62
63         while (true) {
64             printHeader(APP_TITLE + " - KASIR/ADMIN");
65             printMenuOption("1", "Kelola Film");
66             printMenuOption("2", "Transaksi Tiket Baru");
67             printMenuOption("3", "Riwayat Transaksi");
68             printMenuOption("4", "Laporan Transaksi");
69             printMenuOption("5", "Cari / Urut Film");
70             printMenuOption("0", "Keluar");
71

```

```

71
72     String pilih = readTextWithCmd("Pilih menu: ");
73     switch (pilih) {
74         case "1" -> menuKelolaFilm();
75         case "2" -> transaksiTiketBaru();
76         case "3" -> menuRiwayatTransaksi();
77         case "4" -> menuLaporan();
78         case "5" -> menuCariUrutFilm();
79         case "0" -> {
80             saveData();
81             success("Data disimpan. Program selesai.");
82             return;
83         }
84         default -> error("Pilihan tidak valid.");
85     }
86 }
87

```

3.2.3 Menu Kelola Film

Menu Kelola Film menyediakan sub-menu untuk operasi CRUD lengkap. Setiap pilihan memanggil metode yang sesuai dan kembali ke menu ini setelah selesai.

SistemBioskop.Java - Bagian 3: menuKelolaFilm()

```

92     private static void menuKelolaFilm() {
93         while (true) {
94             printHeader("KELOLA FILM");
95             printMenuOption("1", "Lihat Daftar Film");
96             printMenuOption("2", "Tambah Film");
97             printMenuOption("3", "Edit Film");
98             printMenuOption("4", "Hapus Film (Soft Delete)");
99             printMenuOption("5", "Kelola Jadwal Tayang");
100            printMenuOption("0", "Kembali");
101
102            String pilih = readTextWithCmd("Pilih menu: ");
103            switch (pilih) {
104                case "1" -> tampilkanDaftarFilm(true);
105                case "2" -> tambahFilm();
106                case "3" -> editFilm();
107                case "4" -> hapusFilm();
108                case "5" -> kelolaJadwalTayang();
109                case "0" -> { return; }
110                default -> error("Pilihan tidak valid.");
111            }
112        }
113    }

```

3.2.4 Tambah Film (Create)

Metode `tambahFilm()` memvalidasi kode film dengan regex `^F\d{3}$`, memeriksa keunikan kode, lalu mengumpulkan semua atribut film sebelum menambahkan objek Film baru ke `daftarFilm` dan menyimpannya ke CSV.

SistemBioskop.Java - Bagian 4: `tambahFilm()`

```
593 private static void tambahFilm() {
594     printHeader("TAMBAH FILM");
595
596     String kode;
597     while (true) {
598         kode = readTextWithCmd("Kode film (format F001) [C batal]: ");
599         if (isCancel(kode) || isBack(kode)) return;
600         if (!validFilmId(kode)) {
601             error("Format kode film harus F diikuti 3 digit, contoh F001.");
602             continue;
603         }
604         if (cariFilmByKode(kode) != null) {
605             error("Kode film sudah dipakai.");
606             continue;
607         }
608         break;
609     }
610
611     String judul = readNonEmptyWithBackCancel("Judul film [B kembali, C batal]: ");
612     if (judul == null) return;
613
614     String genre = readNonEmptyWithBackCancel("Genre film [B kembali, C batal]: ");
615     if (genre == null) return;
616
617     Integer studio = readIntWithBackCancel("Studio [B kembali, C batal]: ", 1, 99);
618     if (studio == null) return;
619
620     Integer durasi = readIntWithBackCancel("Durasi film (menit) [B kembali, C batal]: ", 1, 1000);
621     if (durasi == null) return;
622
623     Double harga = readDoubleWithBackCancel("Harga tiket [B kembali, C batal]: ", 1, 1000000000);
624     if (harga == null) return;
625
626     Integer jumlahJadwal = readIntWithBackCancel("Jumlah jadwal tayang (1-5) [B kembali, C batal]: ", 1, 5);
627     if (jumlahJadwal == null) return;
628
629     List<JadwalTayang> jadwalBaru = new ArrayList<>();
630     int idx = 1;
631     while (idx <= jumlahJadwal) {
632         String waktu = readTextWithCmd("Jam mulai jadwal ke-" + idx + " (HH:mm) [B kembali, C batal]: ");
633         if (isCancel(waktu)) return;
634         if (isBack(waktu)) {
635             if (!jadwalBaru.isEmpty()) {
636                 jadwalBaru.remove(jadwalBaru.size() - 1);
637                 idx--;
638             } else {
639                 return;
640             }
641             continue;
642         }
643         jadwalBaru.add(new JadwalTayang(waktu));
644         idx++;
645     }
646 }
```

```

642     }
643     if (!validTime(waktu)) {
644         error("Format jam harus HH:mm, contoh 19:00.");
645         continue;
646     }
647     if (jadwalSudahAda(jadwalBaru, waktu)) {
648         error("Jam sudah dimasukkan.");
649         continue;
650     }
651     jadwalBaru.add(new JadwalTayang(waktu));
652     idx++;
653 }
654
655 Film film = new Film(kode.toUpperCase(Locale.ROOT), judul, genre, studio, durasi, harga, true);
656 film.jadwal.addAll(jadwalBaru);
657 daftarFilm.add(film);
658 saveData();
659 success("Film berhasil ditambahkan.");
660 }

```

3.2.5 Tampilkan Daftar Film (Read)

Metode tampilkanDaftarFilm() menampilkan semua film dalam format tabel terstruktur. Parameter aktifSaja mengontrol apakah film yang sudah di-soft delete ikut ditampilkan.

SistemBioskop.Java - Bagian 5: tampilkanDaftarFilm()

```

542 private static void tampilkanDaftarFilm(boolean aktifSaja) {
543     tampilkanDaftarFilm(daftarFilm, aktifSaja);
544 }
545
546 private static void tampilkanDaftarFilm(List<Film> sumberFilm, boolean aktifSaja) {
547     //System.out.println("          DAFTAR FILM");
548     System.out.println(MAGENTA + "-----DAFTAR FILM-----" + RESET);
549     //System.out.println("=====");
550     //System.out.println("          TRANSAKSI TIKET BARU");
551     //printSection("DAFTAR FILM");
552     if (sumberFilm.isEmpty()) {
553         warn("Belum ada film.");
554         return;
555     }
556
557     int no = 1;
558     boolean adaData = false;
559     System.out.printf("%-3s %-6s %-28s %-12s %-7s %-12s\n",
560         "No", "Kode", "Judul", "Genre", "Studio", "Durasi", "Status");
561     System.out.println(repeat("-", UI_WIDTH));
562     for (Film film : sumberFilm) {
563         if (aktifSaja && !film.aktif) continue;
564         adaData = true;
565         System.out.printf("%-3d %-6s %-28s %-12s %-7d %-7s %-12s\n",
566             no++,
567             film.kodeFilm,
568             truncate(film.judul, 28),
569             truncate(film.genre, 12),
570             film.studio,
571             film.durasiMenit + "m",
572             film.aktif ? "Aktif" : "Nonaktif");

```

```

573         System.out.println("    Harga    : Rp" + formatRupiah(film.hargaTiket)
574             + " | Terjual: " + film.tiketTerjual + " tiket");
575         if (!film.jadwal.isEmpty()) {
576             System.out.print("    Jadwal    : ");
577             for (int i = 0; i < film.jadwal.size(); i++) {
578                 JadwalTayang j = film.jadwal.get(i);
579                 System.out.print(j.jamMulai + " (" + hitungKursiTersedia(j) + " kursi)");
580                 if (i < film.jadwal.size() - 1) System.out.print(", ");
581             }
582             System.out.println();
583         } else {
584             System.out.println("    Jadwal    : belum ada");
585         }
586         System.out.println();
587     }
588     if (!adaData) {
589         warn("Tidak ada film yang sesuai filter.");
590     }
591 }

```

3.2.6 Edit Film (Update) Pola Temporary Object

Fitur edit menggunakan pola temporary object: perubahan diterapkan ke salinan Film (Film temp = film.copy()), bukan langsung ke objek asli. Data asli hanya diperbarui setelah pengguna memilih 'Simpan perubahan', sehingga pembatalan edit tidak merusak data.

SistemBioskop.Java - Bagian 6: editFilm()

```

662 private static void editFilm() {
663     printHeader("EDIT FILM");
664     tampilkanDaftarFilm(true);
665
666     String kode = readTextWithCmd("Masukkan kode film yang diedit [C batal]: ");
667     if (isCancel(kode) || isBack(kode)) return;
668     Film film = cariFilmByKodeAktif(kode);
669     if (film == null) {
670         error("Film tidak ditemukan atau tidak aktif.");
671         return;
672     }
673
674     Film temp = film.copy();
675     while (true) {
676         System.out.println();
677         printSection("DATA FILM SAAT INI");
678         printMenuOption("1", "Judul    : " + temp.judul);
679         printMenuOption("2", "Genre    : " + temp.genre);
680         printMenuOption("3", "Studio   : " + temp.studio);
681         printMenuOption("4", "Durasi   : " + temp.durasiMenit + " menit");
682         printMenuOption("5", "Harga    : Rp" + formatRupiah(temp.hargaTiket));
683         printMenuOption("6", "Jadwal   : " + temp.jadwal.size() + " jadwal");
684         printMenuOption("7", "Simpan perubahan");
685         printMenuOption("0", "Batal edit");
686         String pilih = readTextWithCmd("Pilih field yang ingin diubah: ");
687         if (isCancel(pilih)) return;
688
689         switch (pilih) {
690             case "1" -> {
691                 String v = readNonEmptyWithBackCancel("Judul baru [B kembali, C batal]: ");
692                 if (v == null) return;
693                 temp.judul = v;
694             }

```

```

695         case "2" -> {
696             String v = readNonEmptyWithBackCancel("Genre baru [B kembali, C batal]: ");
697             if (v == null) return;
698             temp.genre = v;
699         }
700         case "3" -> {
701             Integer v = readIntWithBackCancel("Studio baru [B kembali, C batal]: ", 1, 99);
702             if (v == null) return;
703             temp.studio = v;
704         }
705         case "4" -> {
706             Integer v = readIntWithBackCancel("Durasi baru (menit) [B kembali, C batal]: ", 1, 1000);
707             if (v == null) return;
708             temp.durasiMenit = v;
709         }
710         case "5" -> {
711             Double v = readDoubleWithBackCancel("Harga baru [B kembali, C batal]: ", 1, 1000000000);
712             if (v == null) return;
713             temp.hargaTiket = v;
714         }
715         case "6" -> editJadwalFilm(temp, false);
716         case "7" -> {
717             film.judul = temp.judul;
718             film.genre = temp.genre;
719             film.studio = temp.studio;
720             film.durasiMenit = temp.durasiMenit;
721             film.hargaTiket = temp.hargaTiket;
722             film.jadwal = temp.jadwal;
723             saveData();
724             success("Perubahan film berhasil disimpan.");
725             return;
726         }
727
728         case "0" -> {
729             warn("Edit dibatalkan.");
730             return;
731         }
732         default -> error("Pilihan tidak valid.");
733     }
734 }
735

```

3.2.7 Kelola Jadwal Tayang

Metode `editJadwalFilm()` mengelola tambah, ubah, dan hapus jadwal. Jadwal hanya boleh diubah atau dihapus jika belum ada kursi yang terisi (`countOccupied() == 0`). Parameter `autoSave` mengontrol apakah perubahan langsung disimpan ke CSV atau ditunda sampai konfirmasi di mode edit film.

SistemBioskop.Java - Bagian 7: `editJadwalFilm()`

```

736 private static void editJadwalFilm(Film film) {
737     editJadwalFilm(film, true);
738 }
739
740 private static void editJadwalFilm(Film film, boolean autoSave) {
741     while (true) {
742         System.out.println();
743         printSection("JADWAL FILM");
744         if (film.jadwal.isEmpty()) {
745             System.out.println("Belum ada jadwal.");
746         } else {
747             for (int i = 0; i < film.jadwal.size(); i++) {
748                 JadwalTayang j = film.jadwal.get(i);
749                 System.out.println((i + 1) + ". " + j.jamMulai + " - "
750                     + j.getJamSelesai(film.durasiMenit).format(TIME_FMT)
751                     + " | Terisi: " + j.countOccupied() + "/" + (ROWS * COLS));
752             }
753         }
754         if (!autoSave) {
755             info("Perubahan jadwal di mode edit film disimpan setelah memilih 'Simpan perubahan.'");
756         }
757         printMenuOption("1", "Tambah jadwal");
758         printMenuOption("2", "Ubah jadwal kosong");
759         printMenuOption("3", "Hapus jadwal kosong");
760         printMenuOption("0", "Kembali");
761         String pilih = readTextWithCmd("Pilih menu: ");
762         if (isCancel(pilih) || "0".equals(pilih)) return;
763     }

```

```

764     switch (pilih) {
765         case "1" -> {
766             String waktu = readTextWithCmd("Jam mulai baru (HH:mm) [B kembali, C batal]: ");
767             if (isCancel(waktu) || isBack(waktu)) continue;
768             if (!validTime(waktu)) {
769                 error("Format jam harus HH:mm.");
770                 continue;
771             }
772             boolean duplikat = jadwalSudahAda(film.jadwal, waktu);
773             if (duplikat) {
774                 error("Jam sudah ada.");
775                 continue;
776             }
777             film.jadwal.add(new JadwalTayang(waktu));
778             saveIfNeeded(autoSave);
779             success(autoSave ? "Jadwal berhasil ditambahkan." : "Jadwal sementara berhasil ditambahkan.");
780         }
781         case "2" -> {
782             if (film.jadwal.isEmpty()) {
783                 warn("Tidak ada jadwal untuk diubah.");
784                 continue;
785             }
786             Integer idx = readIntWithBackCancel("Nomor jadwal yang diubah [B kembali, C batal]: ", 1, film.jadwal.size());
787             if (idx == null) return;
788             JadwalTayang target = film.jadwal.get(idx - 1);
789             if (target.countOccupied() > 0) {
790                 warn("Jadwal sudah memiliki kursi terisi, jadi tidak boleh diubah.");
791                 continue;
792             }
793             String waktu = readTextWithCmd("Jam baru (HH:mm) [B kembali, C batal]: ");

```

```

794             if (isCancel(waktu) || isBack(waktu)) continue;
795             if (!validTime(waktu)) {
796                 error("Format jam harus HH:mm.");
797                 continue;
798             }
799             boolean duplikat = false;
800             for (int i = 0; i < film.jadwal.size(); i++) {
801                 if (i != idx - 1 && film.jadwal.get(i).jamMulai.equals(waktu)) {
802                     duplikat = true;
803                     break;
804                 }
805             }
806             if (duplikat) {
807                 error("Jam sudah ada.");
808                 continue;
809             }
810             target.jamMulai = waktu;
811             saveIfNeeded(autoSave);
812             success(autoSave ? "Jadwal berhasil diubah." : "Jadwal sementara berhasil diubah.");
813         }
814     }

```



```

814     case "3" -> {
815         if (film.jadwal.isEmpty()) {
816             warn("Tidak ada jadwal untuk dihapus.");
817             continue;
818         }
819         Integer idx = readIntWithBackCancel("Nomor jadwal yang dihapus [B kembali, C batal]: ", 1, film.jadwal.size());
820         if (idx == null) return;
821         JadwalTayang target = film.jadwal.get(idx - 1);
822         if (target.countOccupied() > 0) {
823             warn("Jadwal sudah memiliki kursi terisi, jadi tidak boleh dihapus.");
824             continue;
825         }
826         film.jadwal.remove((int) idx - 1);
827         saveIfNeeded(autoSave);
828         success(autoSave ? "Jadwal berhasil dihapus." : "Jadwal sementara berhasil dihapus.");
829     }
830     default -> error("Pilihan tidak valid.");
831 }
832 }
833 }

```

3.2.8 Hapus Film (Soft Delete)

Penghapusan film hanya mengubah atribut aktif menjadi false, tanpa menghapus baris dari file CSV. Ini menjaga integritas referensi data transaksi lama yang mengacu pada film tersebut.

SistemBioskop.Java - Bagian 8: hapusFilm() soft Delete

```

835     private static void hapusFilm() {
836         printHeader("HAPUS FILM");
837         tampilkanDaftarFilm(true);
838         String kode = readTextWithCmd("Masukkan kode film yang ingin dihapus [C batal]: ");
839         if (isCancel(kode) || isBack(kode)) return;
840         Film film = cariFilmByKodeAktif(kode);
841         if (film == null) {
842             error("Film tidak ditemukan atau sudah nonaktif.");
843             return;
844         }
845         film.aktif = false;
846         saveData();
847         success("Film berhasil dihapus (soft delete).");
848     }
849
850     private static void kelolaJadwalTayang() {
851         printHeader("KELOLA JADWAL TAYANG");
852         tampilkanDaftarFilm(true);
853         String kode = readTextWithCmd("Masukkan kode film [C batal]: ");
854         if (isCancel(kode) || isBack(kode)) return;
855         Film film = cariFilmByKodeAktif(kode);
856         if (film == null) {
857             error("Film tidak ditemukan atau tidak aktif.");
858             return;
859         }
860         editJadwalFilm(film);
861     }

```

3.2.9 Algoritma Pencarian Manual

Sistem mengimplementasikan tiga jenis pencarian manual: (1) Linear Search substring untuk keyword film, (2) Sequential Search untuk ID transaksi, dan (3) Filter berdasarkan kategori genre.

SistemBioskop.Java - Bagian 9: Algoritma pencarian Manual

```
306 private static boolean containsIgnoreCaseManual(String sumber, String kataKunci) {
307     if (sumber == null || kataKunci == null) return false;
308     String teks = sumber.toLowerCase(Locale.ROOT);
309     String dicari = kataKunci.toLowerCase(Locale.ROOT);
310     if (dicari.isEmpty()) return true;
311     if (dicari.length() > teks.length()) return false;
312
313     for (int i = 0; i <= teks.length() - dicari.length(); i++) {
314         boolean cocok = true;
315         for (int j = 0; j < dicari.length(); j++) {
316             if (teks.charAt(i + j) != dicari.charAt(j)) {
317                 cocok = false;
318                 break;
319             }
320         }
321         if (cocok) return true;
322     }
323     return false;
324 }
```

```
194 private static void cariFilmByKeyword() {
195     String keyword = readTextWithCmd("Masukkan kata kunci [C batal]: ");
196     if (isCancel(keyword) || isBack(keyword)) return;
197     String key = keyword.trim().toLowerCase(Locale.ROOT);
198     if (key.isEmpty()) {
199         warn("Kata kunci tidak boleh kosong.");
200         return;
201     }
202
203     List<Film> hasil = new ArrayList<>();
204     for (Film film : daftarFilm) {
205         if (!film.aktif) continue;
206         boolean cocok = containsIgnoreCaseManual(film.kodeFilm, key)
207             || containsIgnoreCaseManual(film.judul, key)
208             || containsIgnoreCaseManual(film.genre, key);
209         if (cocok) hasil.add(film);
210     }
211     printSection("HASIL PENCARIAN");
212     tampilkanDaftarFilm(hasil, true);
213 }
214 }
```

```

215 private static void filterFilmByGenre() {
216     Map<String, Integer> genreCount = new LinkedHashMap<>();
217     for (Film film : daftarFilm) {
218         if (!film.aktif) continue;
219         genreCount.put(film.genre, genreCount.getOrDefault(film.genre, 0) + 1);
220     }
221     if (genreCount.isEmpty()) {
222         warn("Belum ada film aktif.");
223         return;
224     }
225
226     printSection("GENRE TERSEDIA");
227     int no = 1;
228     for (Map.Entry<String, Integer> entry : genreCount.entrySet()) {
229         System.out.printf("%2d. %-20s %d film%n", no++, entry.getKey(), entry.getValue());
230     }
231
232     String genre = readTextWithCmd("Masukkan genre [C batal]: ");
233     if (isCancel(genre) || isBack(genre)) return;
234     String key = genre.trim().toLowerCase(Locale.ROOT);
235     if (key.isEmpty()) {
236         warn("Genre tidak boleh kosong.");
237         return;
238     }
239
240     List<Film> hasil = new ArrayList<>();
241     for (Film film : daftarFilm) {
242         if (film.aktif && containsIgnoreCaseManual(film.genre, key)) {
243             hasil.add(film);
244         }
245     }
246     printSection("HASIL FILTER GENRE");
247     tampilkanDaftarFilm(hasil, true);
248 }

```

```

994 private static Film cariFilmByKode(String kode) {
995     if (kode == null) return null;
996     String key = kode.trim().toUpperCase(Locale.ROOT);
997     for (Film film : daftarFilm) {
998         if (film.kodeFilm.equalsIgnoreCase(key)) {
999             return film;
1000         }
1001     }
1002     return null;
1003 }
1004
1005 private static Film cariFilmByKodeAktif(String kode) {
1006     Film film = cariFilmByKode(kode);
1007     if (film != null && film.aktif) return film;
1008     return null;
1009 }

```

3.2.10 Algoritma Pengurutan Manual (Selection Sort)

Pengurutan diimplementasikan secara manual menggunakan algoritma Selection Sort tanpa menggunakan `Collections.sort()` atau `Comparator` bawaan Java. Terdapat tiga mode pengurutan: berdasarkan kode (ascending), judul (alphabetical), dan tiket terjual (descending).

SistemBioskop.Java - Bagian 10: Selection sort Manual (3 mode)

```
250 private static void tampilkanFilmTerurut(String judul, int modeUrut) {
251     List<Film> hasil = new ArrayList<>();
252     for (Film film : daftarFilm) {
253         if (film.aktif) hasil.add(film);
254     }
255     urutkanFilmManual(hasil, modeUrut);
256     printSection(judul);
257     tampilkanDaftarFilm(hasil, true);
258 }
259
260 private static void urutkanFilmManual(List<Film> data, int modeUrut) {
261     for (int i = 0; i < data.size() - 1; i++) {
262         int indeksPilihan = i;
263         for (int j = i + 1; j < data.size(); j++) {
264             if (perluTukarUrutan(data.get(indeksPilihan), data.get(j), modeUrut)) {
265                 indeksPilihan = j;
266             }
267         }
268         if (indeksPilihan != i) {
269             Film sementara = data.get(i);
270             data.set(i, data.get(indeksPilihan));
271             data.set(indeksPilihan, sementara);
272         }
273     }
274 }
```

```
276 private static boolean perluTukarUrutan(Film filmSaatIni, Film filmKandidat, int modeUrut) {
277     if (modeUrut == 1) {
278         return bandingkanTextManual(filmSaatIni.kodeFilm, filmKandidat.kodeFilm) > 0;
279     }
280     if (modeUrut == 2) {
281         return bandingkanTextManual(filmSaatIni.judul, filmKandidat.judul) > 0;
282     }
283     if (modeUrut == 3) {
284         if (filmKandidat.tiketTerjual != filmSaatIni.tiketTerjual) {
285             return filmKandidat.tiketTerjual > filmSaatIni.tiketTerjual;
286         }
287         return bandingkanTextManual(filmSaatIni.judul, filmKandidat.judul) > 0;
288     }
289     return false;
290 }
```

```
292 private static int bandingkanTextManual(String kiri, String kanan) {
293     String a = kiri == null ? "" : kiri.toLowerCase(Locale.ROOT);
294     String b = kanan == null ? "" : kanan.toLowerCase(Locale.ROOT);
295     int batas = Math.min(a.length(), b.length());
296     for (int i = 0; i < batas; i++) {
297         char ca = a.charAt(i);
298         char cb = b.charAt(i);
299         if (ca != cb) {
300             return ca - cb;
301         }
302     }
303     return a.length() - b.length();
304 }
305
```


3.2.11 Alur Transaksi Tiket Baru (State Machine 7 Tahap)

Fitur inti sistem diimplementasikan sebagai state machine berbasis variabel integer step. Setiap nilai step merepresentasikan satu tahapan dalam alur transaksi. Pengguna dapat navigasi mundur (B) ke tahap sebelumnya atau membatalkan seluruh transaksi (C) di mana saja.

SistemBioskop.Java - Bagian 11: transaksiTiketBaru() state Machine

```

329 private static void transaksiTiketBaru() {
330     if (!adaFilmAktif()) {
331         warn("Tidak ada film aktif.");
332         return;
333     }
334
335     printHeader("TRANSAKSI TIKET BARU");
336
337     String tanggal = readTanggal("Masukkan tanggal transaksi (dd/MM/yyyy) [C batal]: ");
338     if (tanggal == null) return;
339
340     Film film = null;
341     JadwalTayang jadwal = null;
342     int jumlahTiket = 0;
343     List<String> kursiDipilih = new ArrayList<>();
344
345     int step = 1;
346     while (true) {
347         if (step == 1) {
348             tampilkanDaftarFilm(true);
349             String kode = readTextWithCmd("Pilih kode film [C batal]: ");
350             if (isCancel(kode)) {
351                 warn("Transaksi dibatalkan.");
352                 return;
353             }
354             film = cariFilmByKodeAktif(kode);
355             if (film == null) {
356                 error("Film tidak ditemukan atau tidak aktif.");
357                 continue;
358             }
359             if (film.jadwal.isEmpty()) {
360                 warn("Film ini belum memiliki jadwal tayang.");
361                 continue;
362             }
363
364             // ... (this part continues from the previous block)
365         }
366
367         if (step == 2) {
368             tampilkanJadwalFilm(film);
369             String pil = readTextWithCmd("Pilih nomor jadwal [B kembali, C batal]: ");
370             if (isCancel(pil)) {
371                 warn("Transaksi dibatalkan.");
372                 return;
373             }

```

```

374         if (isBack(pil)) {
375             step = 1;
376             continue;
377         }
378         Integer idx = parseIntOrNull(pil);
379         if (idx == null || idx < 1 || idx > film.jadwal.size()) {
380             error("Pilihan jadwal tidak valid.");
381             continue;
382         }
383         jadwal = film.jadwal.get(idx - 1);
384         step = 3;
385         continue;
386     }
387
388     if (step == 3) {
389         int tersedia = hitungKursiTersedia(jadwal);
390         String q = readTextWithCmd("Masukkan jumlah tiket [B kembali, C batal]: ");
391         if (isCancel(q)) {
392             warn("Transaksi dibatalkan.");
393             return;
394         }
395         if (isBack(q)) {
396             step = 2;
397             continue;
398         }
399         Integer qty = parseIntOrNull(q);
400         if (qty == null || qty < 1) {
401             error("Jumlah tiket harus minimal 1.");
402             continue;
403         }

```

```

404         if (qty > tersedia) {
405             warn("Kursi tersedia hanya " + tersedia + ".");
406             continue;
407         }
408         jumlahTiket = qty;
409         kursiDipilih.clear();
410         step = 4;
411         continue;
412     }
413
414     if (step == 4) {
415         while (kursiDipilih.size() < jumlahTiket) {
416             tampilkanDenahKursi(jadwal, kursiDipilih);
417             System.out.println("Kursi sementara : " + (kursiDipilih.isEmpty() ? "-" : String.join(", ", kursiDipilih)));
418             String seat = readTextWithCmd("Pilih kursi ke-" + (kursiDipilih.size() + 1) + " [B kembali, C batal]: ");
419             if (isCancel(seat)) {
420                 warn("Transaksi dibatalkan.");
421                 return;
422             }
423             if (isBack(seat)) {
424                 kursiDipilih.clear();
425                 step = 3;
426                 break;
427             }
428             seat = seat.toUpperCase(Locale.ROOT);
429             if (!validSeatCode(seat)) {
430                 error("Format kursi salah. Contoh: A1, C10.");
431                 continue;
432             }
433             if (jadwal.isOccupied(seat) || kursiSudahDipilih(kursiDipilih, seat)) {
434                 error("Kursi sudah terisi atau sudah dipilih.");
435                 continue;

```

```

436     }
437     kursiDipilih.add(seat);
438 }
439 if (step == 3) continue;
440 step = 5;
441 continue;
442 }
443
444 if (step == 5) {
445     double hargaPerTiket = film.hargaTiket;
446     double total = hargaPerTiket * jumlahTiket;
447     LocalTime jamMulai = jadwal.getJamMulai();
448     LocalTime jamSelesai = jadwal.getJamSelesai(film.durasiMenit);
449
450     System.out.println();
451     printSection("RINGKASAN TRANSAKSI");
452     System.out.println(label("Tanggal") + tanggal);
453     System.out.println(label("Film") + film.judul);
454     System.out.println(label("Genre") + film.genre);
455     System.out.println(label("Studio") + film.studio);
456     System.out.println(label("Jam mulai") + jamMulai.format(TIME_FMT));
457     System.out.println(label("Jam selesai") + jamSelesai.format(TIME_FMT));
458     System.out.println(label("Jumlah tiket") + jumlahTiket);
459     System.out.println(label("Kursi") + String.join(", ", kursiDipilih));
460     System.out.println(label("Harga/tiket") + "Rp" + formatRupiah(hargaPerTiket));
461     System.out.println(label("Total") + color("Rp" + formatRupiah(total), GREEN + BOLD));
462     System.out.println();
463
464     String lanjut = readTextWithCmd("Tekan ENTER untuk lanjut ke pembayaran, atau [B kembali, C batal]: ");
465     if (isCancel(lanjut)) {
466         warn("Transaksi dibatalkan.");
467         return;

```

```

468     }
469     if (isBack(lanjut)) {
470         kursiDipilih.clear();
471         step = 4;
472         continue;
473     }
474     step = 6;
475     continue;
476 }
477
478 if (step == 6) {
479     double hargaPerTiket = film.hargaTiket;
480     double total = hargaPerTiket * jumlahTiket;
481     double uangBayar;
482     while (true) {
483         String bayarTxt = readTextWithCmd("Masukkan uang pembayaran [B kembali, C batal]: Rp");
484         if (isCancel(bayarTxt)) {
485             warn("Transaksi dibatalkan.");
486             return;
487         }
488         if (isBack(bayarTxt)) {
489             step = 5;
490             break;
491         }
492         Double bayar = parseDoubleOrNull(bayarTxt);
493         if (bayar == null || bayar < total) {
494             error("Uang tidak mencukupi. Minimal Rp" + formatRupiah(total));
495             continue;
496         }
497         uangBayar = bayar;
498         double kembalian = uangBayar - total;

```

```

499         step = 7;
500
501         String idTrans = generateNextTransactionId();
502         String jamMulaiStr = jadwal.getJamMulai().format(TIME_FMT);
503         String jamSelesaiStr = jadwal.getJamSelesai(film.durasiMenit).format(TIME_FMT);
504         Transaksi trx = new Transaksi(

```



```

505         idTrans,
506         tanggal,
507         film.kodeFilm,
508         film.judul,
509         film.genre,
510         film.studio,
511         jamMulaiStr,
512         jamSelesaiStr,
513         film.durasiMenit,
514         hargaPerTiket,
515         jumlahTiket,
516         new ArrayList<>(kursiDipilih),
517         total,
518         uangBayar,
519         kembalian
520     );
521     daftarTransaksi.add(trx);
522     for (String seat : kursiDipilih) {
523         jadwal.occupy(seat);
524     }

525     film.tiketTerjual += jumlahTiket;
526     saveData();
527     System.out.println();
528     printInvoice(trx);
529     success("Pembayaran berhasil. Kursi sudah dikunci.");
530     return;
531 }
532 if (step == 5) continue;
533 }
534
535 // step 7 is finalization only; no cancel options
536 }
537 }
538

```

3.2.12 Visualisasi Denah Kursi Interaktif

Denah kursi ditampilkan secara real-time menggunakan matriks boolean JadwalTayang. Kursi terisi ditandai [X] (merah) dan kursi tersedia ditandai [] (hijau). Kursi yang baru dipilih dalam sesi ini juga langsung ditandai [X] meski belum tersimpan.

SistemBioskop.Java - Bagian 12: tampilkanDenahKursi()

```

1039 private static void tampilkanDenahKursi(JadwalTayang jadwal, List<String> kursiDipilih) {
1040     System.out.println();
1041     System.out.println(color("----- LAYAR -----", BLUE + BOLD));
1042     System.out.print("    ");
1043     for (int c = 1; c <= COLS; c++) {
1044         System.out.print(String.format("%4d", c));
1045     }
1046     System.out.println();
1047     for (int r = 0; r < ROWS; r++) {
1048         char row = (char) ('A' + r);
1049         System.out.print(row + "    ");
1050         for (int c = 0; c < COLS; c++) {
1051             String code = "" + row + (c + 1);
1052             boolean terisi = jadwal.isOccupied(code) || kursiSudahDipilih(kursiDipilih, code);
1053             System.out.print(terisi ? color(" [X]", RED) : color(" [ ]", GREEN));
1054         }
1055         System.out.println();
1056     }
1057     System.out.println("Keterangan: " + color("[ ]", GREEN) + " Tersedia, " + color("[X]", RED) + " Terisi / Dipilih");
1058 }
1059
1060 private static boolean adaFilmAktif() {
1061     for (Film film : daftarFilm) {
1062         if (film.aktif) {
1063             return true;
1064         }
1065     }
1066     return false;
1067 }

```

```

1077
1078 private static boolean kursiSudahDipilih(List<String> daftarKursi, String kursi) {
1079     for (String item : daftarKursi) {
1080         if (item.equalsIgnoreCase(kursi)) {
1081             return true;
1082         }
1083     }
1084     return false;
1085 }
1086
1087 private static boolean validSeatCode(String seat) {
1088     if (seat == null) return false;
1089     seat = seat.trim().toUpperCase(Locale.ROOT);
1090     return seat.matches("^[A-E](10|[1-9])$");
1091 }
1092
1093 private static boolean validFilmId(String id) {
1094     return id != null && id.trim().toUpperCase(Locale.ROOT).matches("^F\\d{3}$");
1095 }
1096

```

3.2.13 Riwayat Transaksi & Laporan Statistik

Metode `menuLaporan()` mengagregasikan seluruh data transaksi secara real-time menggunakan `HashMap` untuk menghitung tiket per film, kemudian menelusuri `daftarFilm` secara linear untuk menemukan film terlaris.

SistemBioskop.Java - Bagian 13: `menuRiwayatTransaksi()`, `menuLaporan()`, `printInvoice()`

```

118 ... private static void menuRiwayatTransaksi() {
119     while (true) {
120         printHeader("RIWAYAT TRANSAKSI");
121         printMenuOption("1", "Lihat Semua Transaksi");
122         printMenuOption("2", "Cari Transaksi by ID");
123         printMenuOption("0", "Kembali");
124
125         String pilih = readTextWithCmd("Pilih menu: ");
126         switch (pilih) {
127             case "1" -> tampilkanRiwayatTransaksi();
128             case "2" -> cariTransaksiById();
129             case "0" -> { return; }
130             default -> error("Pilihan tidak valid.");
131         }
132     }
133 }

```

```

866 ... private static void tampilkanRiwayatTransaksi() {
867     printHeader("RIWAYAT TRANSAKSI");
868     if (daftarTransaksi.isEmpty()) {
869         warn("Belum ada transaksi.");
870         return;
871     }
872     System.out.printf("%-4s %-8s %-12s %-6s %-8s %-18s %-12s%n",
873         "No", "ID", "Tanggal", "Film", "Jam", "Kursi", "Total");
874     System.out.println(repeat("-", UI_WIDTH));
875     for (int i = 0; i < daftarTransaksi.size(); i++) {
876         Transaksi t = daftarTransaksi.get(i);
877         System.out.printf("%-4d %-8s %-12s %-6s %-8s %-18s Rp%-10s%n",
878             i + 1,
879             t.idTransaksi,
880             t.tanggal,
881             t.kodeFilm,
882             t.jamMulai,
883             truncate(String.join(", ", t.kursiDipilih), 18),
884             formatRupiah(t.totalBayar));
885     }
886 }

```

```

888     private static void cariTransaksiById() {
889         String kode = readTextWithCmd("Masukkan ID transaksi (format TRX001) [C batal]: ");
890         if (isCancel(kode) || isBack(kode)) return;
891         if (!validTransaksiId(kode)) {
892             error("Format ID transaksi tidak valid.");
893             return;
894         }
895         Transaksi t = cariTransaksiByIdInternal(kode);
896         if (t == null) {
897             error("Transaksi tidak ditemukan.");
898             return;
899         }
900         printInvoice(t);
901     }

```

```

135     private static void menuLaporan() {
136         printHeader("LAPORAN TRANSAKSI");
137
138         int totalTransaksi = daftarTransaksi.size();
139         int totalTiket = 0;
140         double totalPendapatan = 0;
141
142         Map<String, Integer> tiketPerFilm = new HashMap<>();
143         for (Transaksi t : daftarTransaksi) {
144             totalTiket += t.jumlahTiket;
145             totalPendapatan += t.totalBayar;
146             tiketPerFilm.put(t.kodeFilm, tiketPerFilm.getOrDefault(t.kodeFilm, 0) + t.jumlahTiket);
147         }
148
149         Film filmTerlaris = null;
150         int maxTiket = 0;
151         for (Film f : daftarFilm) {
152             int jumlah = tiketPerFilm.getOrDefault(f.kodeFilm, 0);
153             if (filmTerlaris == null || jumlah > maxTiket) {
154                 filmTerlaris = f;
155                 maxTiket = jumlah;
156             }
157         }
158
159         System.out.println(label("Total transaksi") + totalTransaksi);
160         System.out.println(label("Total tiket") + totalTiket);
161         System.out.println(label("Pendapatan") + "Rp" + formatRupiah(totalPendapatan));
162         if (filmTerlaris != null && maxTiket > 0) {
163             System.out.println(label("Film terlaris") + filmTerlaris.judul + " (" + maxTiket + " tiket)");
164         } else {
165             System.out.println(label("Film terlaris") + "belum ada data.");
166         }
167     }

```

```

903     private static void printInvoice(Transaksi t) {
904         System.out.println();
905         printHeader("INVOICE");
906         System.out.println(label("ID Transaksi") + t.idTransaksi);
907         System.out.println(label("Tanggal") + t.tanggal);
908         System.out.println(label("Film") + t.judulFilm);
909         System.out.println(label("Genre") + t.genreFilm);
910         System.out.println(label("Studio") + t.studio);
911         System.out.println(label("Jam mulai") + t.jamMulai);
912         System.out.println(label("Jam selesai") + t.jamSelesai);
913         System.out.println(label("Durasi") + t.durasiFilm + " menit");
914         System.out.println(label("Kursi") + String.join(", ", t.kursiDipilih));
915         System.out.println(label("Jumlah tiket") + t.jumlahTiket);
916         System.out.println(label("Harga/tiket") + "Rp" + formatRupiah(t.hargaPerTiket));
917         System.out.println(label("Total bayar") + color("Rp" + formatRupiah(t.totalBayar), GREEN + BOLD));
918         System.out.println(label("Tunai") + "Rp" + formatRupiah(t.uangBayar));
919         System.out.println(label("Kembalian") + "Rp" + formatRupiah(t.kembalian));
920         System.out.println(repeat("=", UI_WIDTH));
921     }
922

```

3.2.14 Penyimpanan dan Pemuatan Data CSV

Sistem menyimpan data ke dua file CSV terpisah. films.csv menyimpan matriks kursi dalam format bitmap biner 50-karakter (JAM#bitmap). transactions.csv menyimpan snapshot lengkap setiap transaksi, termasuk salinan data film saat transaksi terjadi.

SistemBioskop.Java - Bagian 14: saveData() / loadData() / csv i/o

```
1329     private static void saveData() {
1330         saveFilms();
1331         saveTransactions();
1332     }
1333
1334     private static void saveFilms() {
1335         try (PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(FILM_FILE)))) {
1336             for (Film film : daftarFilm) {
1337                 StringBuilder sb = new StringBuilder();
1338                 sb.append(escape(film.kodeFilm)).append(';')
1339                 .append(escape(film.judul)).append(';')
1340                 .append(escape(film.genre)).append(';')
1341                 .append(film.studio).append(';')
1342                 .append(film.durasiMenit).append(';')
1343                 .append(film.hargaTiket).append(';')
1344                 .append(film.aktif).append(';');
1345                 List<String> scheduleParts = new ArrayList<>();
1346                 for (JadwalTayang j : film.jadwal) {
1347                     scheduleParts.add(j.serialize());
1348                 }
1349                 sb.append(String.join("|", scheduleParts));
1350                 pw.println(sb);
1351             }
1352         } catch (IOException e) {
1353             System.out.println("Gagal menyimpan file film: " + e.getMessage());
1354         }
1355     }
1356 }
```

```

1357 private static void saveTransactions() {
1358     try (PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(TRAN_FILE)))) {
1359         for (Transaksi t : daftarTransaksi) {
1360             pw.println(String.join(";",
1361                 escape(t.idTransaksi),
1362                 escape(t.tanggal),
1363                 escape(t.kodeFilm),
1364                 escape(t.judulFilm),
1365                 escape(t.genreFilm),
1366                 String.valueOf(t.studio),
1367                 escape(t.jamMulai),
1368                 escape(t.jamSelesai),
1369                 String.valueOf(t.durasiFilm),
1370                 String.valueOf(t.hargaPerTiket),
1371                 String.valueOf(t.jumlahTiket),
1372                 escape(String.join(" ", t.kursiDipilih)),
1373                 String.valueOf(t.totalBayar),
1374                 String.valueOf(t.uangBayar),
1375                 String.valueOf(t.kembalian)
1376             ));
1377         }
1378     } catch (IOException e) {
1379         System.out.println("Gagal menyimpan file transaksi: " + e.getMessage());
1380     }
1381 }
1382

```

```

1383 private static void loadData() {
1384     daftarFilm.clear();
1385     daftarTransaksi.clear();
1386     loadFilms();
1387     loadTransactions();
1388     rebuildTicketStats();
1389 }
1390
1391 private static void loadFilms() {
1392     File file = new File(FILM_FILE);
1393     if (!file.exists()) return;
1394     try (BufferedReader br = new BufferedReader(new FileReader(file))) {
1395         String line;
1396         while ((line = br.readLine()) != null) {
1397             if (line.trim().isEmpty()) continue;
1398             String[] parts = line.split(";", -1);
1399             if (parts.length < 7) continue;
1400             Film film = new Film(
1401                 unescape(parts[0]),
1402                 unescape(parts[1]),
1403                 unescape(parts[2]),
1404                 Integer.parseInt(parts[3]),
1405                 Integer.parseInt(parts[4]),
1406                 Double.parseDouble(parts[5]),
1407                 Boolean.parseBoolean(parts[6])
1408             );
1409             if (parts.length >= 8 && !parts[7].isBlank()) {
1410                 String[] jadwalTokens = parts[7].split("\\|");
1411                 for (String token : jadwalTokens) {
1412                     JadwalTayang j = JadwalTayang.deserialize(token);
1413                     if (j != null) film.jadwal.add(j);
1414                 }
1415             }
1416             daftarFilm.add(film);
1417         }
1418     }
1419 }

```

```

1418     } catch (Exception e) {
1419         System.out.println("Gagal memuat film, memakai data default. " + e.getMessage());
1420         daftarFilm.clear();
1421     }
1422 }
1423
1424 private static void loadTransactions() {
1425     File file = new File(TRAN_FILE);
1426     if (!file.exists()) return;
1427     try (BufferedReader br = new BufferedReader(new FileReader(file))) {
1428         String line;
1429         while ((line = br.readLine()) != null) {
1430             if (line.trim().isEmpty()) continue;
1431             String[] parts = line.split(";", -1);
1432             if (parts.length < 15) continue;
1433             List<String> kursi = new ArrayList<>();
1434             String kursiPart = unescape(parts[11]);
1435             if (!kursiPart.isBlank()) kursi.addAll(Arrays.asList(kursiPart.split(",")));
1436             Transaksi t = new Transaksi(

```

```

1437                 unescape(parts[0]),
1438                 unescape(parts[1]),
1439                 unescape(parts[2]),
1440                 unescape(parts[3]),
1441                 unescape(parts[4]),
1442                 Integer.parseInt(parts[5]),
1443                 unescape(parts[6]),
1444                 unescape(parts[7]),
1445                 Integer.parseInt(parts[8]),
1446                 Double.parseDouble(parts[9]),
1447                 Integer.parseInt(parts[10]),
1448                 kursi,
1449                 Double.parseDouble(parts[12]),
1450                 Double.parseDouble(parts[13]),
1451                 Double.parseDouble(parts[14])
1452             );
1453             daftarTransaksi.add(t);
1454         }
1455     } catch (Exception e) {
1456         System.out.println("Gagal memuat transaksi, memulai tanpa riwayat. " + e.getMessage());
1457         daftarTransaksi.clear();
1458     }
1459 }
1460
1461 private static String escape(String s) {
1462     return s == null ? "" : s.replace(";", ",").replace("\n", " ").trim();
1463 }
1464
1465 private static String unescape(String s) {
1466     return s == null ? "" : s.trim();
1467 }
1468

```

3.2.15 Rekonsiliasi Data & Statistik Tiket

Metode `reconcileSeatsFromTransactions()` dijalankan setiap startup untuk memastikan status kursi di memori konsisten dengan riwayat transaksi CSV. Metode `rebuildTicketStats()` menghitung ulang counter tiketTerjual dari file transaksi.

SistemBioskop.Java - Bagian 15: `reconcileSeatssFromTrannsactions()` & `rebuildTicketStats()`

```

1295 private static void reconcileSeatsFromTransactions() {
1296     boolean changed = false;
1297     for (Transaksi t : daftarTransaksi) {
1298         Film film = cariFilmByKode(t.kodeFilm);
1299         if (film == null) continue;
1300         JadwalTayang jadwal = findJadwalByJam(film, t.jamMulai);
1301         if (jadwal == null) continue;
1302         for (String seat : t.kursiDipilih) {
1303             if (!validSeatCode(seat)) continue;
1304             if (!jadwal.isOccupied(seat)) {
1305                 jadwal.occupy(seat);
1306                 changed = true;
1307             }
1308         }
1309     }
1310     if (changed) {
1311         saveFilms();
1312         info("Status kursi diselaraskan dari riwayat transaksi.");
1313     }
1314 }
1315
1316 private static JadwalTayang findJadwalByJam(Film film, String jamMulai) {
1317     if (film == null || jamMulai == null) return null;
1318     for (JadwalTayang jadwal : film.jadwal) {
1319         if (jadwal.jamMulai.equals(jamMulai.trim())) {
1320             return jadwal;
1321         }
1322     }
1323     return null;
1324 }
1325

```

3.2.16 Utilitas Input, Validasi, dan Antarmuka Konsol

Seluruh input pengguna ditangani melalui metode helper yang menggabungkan validasi tipe data, validasi rentang, dan penanganan navigasi (B/C). Normalisasi input uang mendukung berbagai format: 50000, 50.000, 50,000, atau Rp50.000.

SistemBioskop.Java - Bagian 16: Utilitas Input, Validasi, dan Antarmuka

```

1237 private static String readTextWithCmd(String prompt) {
1238     System.out.print(color(prompt, BOLD));
1239     return readLine().trim();
1240 }
1241
1242 private static String readLine() {
1243     try {
1244         if (!input.hasNextLine()) {
1245             System.out.println();
1246             saveData();
1247             info("Input selesai. Program ditutup otomatis.");
1248             System.exit(0);
1249         }
1250         return input.nextLine();
1251     } catch (Exception e) {
1252         return "";
1253     }
1254 }
1255

```



```

1146 private static Integer readIntWithBackCancel(String prompt, int min, int max) {
1147     while (true) {
1148         String s = readTextWithCmd(prompt);
1149         if (isCancel(s) || isBack(s)) return null;
1150         Integer val = parseIntOrNull(s);
1151         if (val == null) {
1152             error("Masukkan angka yang valid.");
1153             continue;
1154         }
1155         if (val < min || val > max) {
1156             error("Nilai harus di antara " + min + " dan " + max + ".");
1157             continue;
1158         }
1159         return val;
1160     }
1161 }
1162
1163 private static Double readDoubleWithBackCancel(String prompt, double min, double max) {
1164     while (true) {
1165         String s = readTextWithCmd(prompt);
1166         if (isCancel(s) || isBack(s)) return null;
1167         Double val = parseDoubleOrNull(s);
1168         if (val == null) {
1169             error("Masukkan angka uang yang valid. Contoh: 50000 atau 50.000.");
1170             continue;
1171         }
1172         if (val < min || val > max) {
1173             error("Nilai harus di antara " + formatRupiah(min) + " dan " + formatRupiah(max) + ".");
1174             continue;
1175         }
1176         return val;
1177     }
1178 }

```

```

1198 private static String normalizeNumberInput(String s) {
1199     if (s == null) return "";
1200     String value = s.trim()
1201         .replaceAll("(?i)rp", "")
1202         .replace(" ", "");
1203     if (value.isEmpty()) return "";
1204
1205     boolean hasDot = containsCharManual(value, '.');
1206     boolean hasComma = containsCharManual(value, ',');
1207     if (hasDot && hasComma) {
1208         int lastDot = value.lastIndexOf('.');
1209         int lastComma = value.lastIndexOf(',');
1210         if (lastComma > lastDot) {
1211             return value.replace(".", "").replace(',', '.');
1212         }
1213         return value.replace(",", "");
1214     }
1215     if (hasDot && value.matches("\\d{1,3}(\\.\\d{3})+")) {
1216         return value.replace(".", "");
1217     }
1218     if (hasComma && value.matches("\\d{1,3}(,\\d{3})+")) {
1219         return value.replace(",", "");
1220     }
1221     if (hasComma) {
1222         return value.replace(',', '.');
1223     }
1224     return value;
1225 }
1226

```

```

1227 private static boolean containsCharManual(String value, char target) {
1228     if (value == null) return false;
1229     for (int i = 0; i < value.length(); i++) {
1230         if (value.charAt(i) == target) {
1231             return true;
1232         }
1233     }
1234     return false;
1235 }
1236

```

```

1188     private static Double parseDoubleOrNull(String s) {
1189         try {
1190             String normalized = normalizeNumberInput(s);
1191             if (normalized.isEmpty()) return null;
1192             return Double.parseDouble(normalized);
1193         } catch (Exception e) {
1194             return null;
1195         }
1196     }

```

```

1093     private static boolean validFilmId(String id) {
1094         return id != null && id.trim().toUpperCase(Locale.ROOT).matches("^F\\d{3}$");
1095     }
1096
1097     private static boolean validTransaksiId(String id) {
1098         return id != null && id.trim().toUpperCase(Locale.ROOT).matches("^TRX\\d{3,}$");
1099     }
1100
1101     private static boolean validTime(String waktu) {
1102         if (waktu == null) return false;
1103         try {
1104             LocalDateTime.parse(waktu.trim(), TIME_FMT);
1105             return true;
1106         } catch (DateTimeParseException e) {
1107             return false;
1108         }
1109     }
1110
1111     private static String readTanggal(String prompt) {
1112         while (true) {
1113             String s = readTextWithCmd(prompt);
1114             if (isCancel(s)) return null;
1115             if (isBack(s)) return null;
1116             if (!validTanggal(s)) {
1117                 error("Format tanggal harus dd/MM/yyyy, contoh 27/01/2026.");
1118                 continue;
1119             }
1120             return s.trim();
1121         }
1122     }
1123

```

```

926     private static void printhead(String title) {
927         System.out.println();
928         System.out.println(color(repeat("=", UI_WIDTH), BLUE));
929         System.out.println(color(center(title, UI_WIDTH), BOLD + CYAN));
930         System.out.println(color(repeat("=", UI_WIDTH), BLUE));
931     }
932
933     private static void printSection(String title) {
934         System.out.println(color("----- " + title + " " + repeat("-", Math.max(1, UI_WIDTH - title.length() - 4)), MAGENTA));
935     }
936
937     private static void printMenuOption(String key, String text) {
938         System.out.println(color(String.format(" %2s", key), YELLOW + BOLD) + ". " + text);
939     }
940
941     private static String label(String label) {
942         return color(String.format("%-14s", label), CYAN) + ": ";
943     }
944
945     private static void success(String message) {
946         System.out.println(color("[OK] " + message, GREEN));
947     }
948
949     private static void warn(String message) {
950         System.out.println(color("[!] " + message, YELLOW));
951     }
952
953     private static void error(String message) {
954         System.out.println(color("[X] " + message, RED));
955     }
956
957     private static void info(String message) {
958         System.out.println(color("[i] " + message, CYAN));
959     }

```

```

961     private static String color(String text, String ansi) {
962         if (!USE_COLOR) return text;
963         return ansi + text + RESET;
964     }
965
966     private static String center(String text, int width) {
967         if (text.length() >= width) return text;
968         int left = (width - text.length()) / 2;
969         return repeat(" ", left) + text;
970     }
971
972     private static String repeat(String text, int count) {
973         StringBuilder sb = new StringBuilder();
974         for (int i = 0; i < count; i++) {
975             sb.append(text);
976         }
977         return sb.toString();
978     }
979
980     private static String truncate(String text, int maxLength) {
981         if (text == null) return "";
982         if (text.length() <= maxLength) return text;
983         if (maxLength <= 3) return text.substring(0, maxLength);
984         return text.substring(0, maxLength - 3) + "...";
985     }
986
987     private static void saveIfNeeded(boolean autoSave) {
988         if (autoSave) saveData();
989     }
990

```

```

1256     private static boolean isCancel(String s) {
1257         return s != null && s.trim().equalsIgnoreCase(CANCEL);
1258     }
1259
1260     private static boolean isBack(String s) {
1261         return s != null && s.trim().equalsIgnoreCase(BACK);
1262     }
1263
1264     private static String formatRupiah(double value) {
1265         return String.format(Locale.ROOT, "%.0f", value).replace(',', '.');
1266     }
1267
1268     private static String generateNextTransactionId() {
1269         int max = 0;
1270         for (Transaksi t : daftarTransaksi) {
1271             String id = t.idTransaksi.toUpperCase(Locale.ROOT);
1272             if (id.startsWith("TRX")) {
1273                 String num = id.substring(3);
1274                 try {
1275                     max = Math.max(max, Integer.parseInt(num));
1276                 } catch (NumberFormatException ignored) {}
1277             }
1278         }
1279         return String.format("TRX%03d", max + 1);
1280     }
1281
1282

```

```

1060     private static boolean adaFilmAktif() {
1061         for (Film film : daftarFilm) {
1062             if (film.aktif) {
1063                 return true;
1064             }
1065         }
1066         return false;
1067     }
1068
1069     private static boolean jadwalSudahAda(List<JadwalTayang> daftarJadwal, String waktu) {
1070         for (JadwalTayang jadwal : daftarJadwal) {
1071             if (jadwal.jamMulai.equals(waktu)) {
1072                 return true;
1073             }
1074         }
1075         return false;
1076     }
1077

```

```

1019 private static void tampilkanJadwalFilm(Film film) {
1020     System.out.println();
1021     System.out.println("----- JADWAL FILM: Tunggu Aku Sukses Nanti -----");
1022     //printSection("JADWAL FILM: " + film.judul);
1023     if (film.jadwal.isEmpty()) {
1024         warn("Belum ada jadwal.");
1025         return;
1026     }
1027     for (int i = 0; i < film.jadwal.size(); i++) {
1028         JadwalTayang j = film.jadwal.get(i);
1029         System.out.println((i + 1) + ". " + j.jamMulai + " - "
1030             + j.getJamSelesai(film.durasiMenit).format(TIME_FMT)
1031             + " | Kursi tersedia: " + hitungKursiTersedia(j));
1032     }
1033 }
1034

```

3.2.17 Data Awal (Seed Data)

Jika films.csv tidak ditemukan saat startup, sistem otomatis membuat 10 film contoh dengan berbagai genre sebagai data demonstrasi.

SistemBioskop.Java - Bagian 17: seedDataAwal ()

```

1472 private static void seedDataAwal() {
1473     Film f1 = new Film("F001", "Tunggu Aku Sukses Nanti", "Drama", 5, 120, 40000, true);
1474     f1.jadwal.add(new JadwalTayang("11:30"));
1475     f1.jadwal.add(new JadwalTayang("14:00"));
1476
1477     Film f2 = new Film("F002", "Senin Harga Naik", "Comedy", 2, 98, 45000, true);
1478     f2.jadwal.add(new JadwalTayang("15:00"));
1479     f2.jadwal.add(new JadwalTayang("17:00"));
1480
1481     Film f3 = new Film("F003", "Ayah, Ini Arahnya Kemana?", "Drama", 1, 103, 50000, true);
1482     f3.jadwal.add(new JadwalTayang("19:00"));
1483     f3.jadwal.add(new JadwalTayang("21:00"));
1484
1485     daftarFilm.add(f1);
1486     daftarFilm.add(f2);
1487     daftarFilm.add(f3);
1488 }
1489

```

3.2.18 Inner Class: Film, JadwalTayang, dan Transaksi

Tiga inner class private static mendefinisikan model data utama sistem. Kelas JadwalTayang memiliki fitur serialisasi/deserialisasi matriks kursi ke format bitmap biner untuk disimpan dalam file CSV.

SistemBioskop.Java - Bagian 18: class Film

```

1493     private static class Film {
1494         String kodeFilm;
1495         String judul;
1496         String genre;
1497         int studio;
1498         int durasiMenit;
1499         double hargaTiket;
1500         boolean aktif;
1501         List<JadwalTayang> jadwal = new ArrayList<>();
1502         int tiketTerjual;
1503
1504         Film(String kodeFilm, String judul, String genre, int studio, int durasiMenit, double hargaTiket, boolean aktif) {
1505             this.kodeFilm = kodeFilm;
1506             this.judul = judul;
1507             this.genre = genre;
1508             this.studio = studio;
1509             this.durasiMenit = durasiMenit;
1510             this.hargaTiket = hargaTiket;
1511             this.aktif = aktif;
1512         }
1513
1514         Film copy() {
1515             Film f = new Film(kodeFilm, judul, genre, studio, durasiMenit, hargaTiket, aktif);
1516             for (JadwalTayang j : jadwal) {
1517                 f.jadwal.add(j.copy());
1518             }
1519             f.tiketTerjual = tiketTerjual;
1520             return f;
1521         }
1522     }
1523

```

SistemBioskop.Java - Bagian 19: class JadwalTayang (matriks Kursi + Serialisasi)

```

1524     private static class JadwalTayang {
1525         String jamMulai;
1526         boolean[][] kursi = new boolean[ROWS][COLS];
1527
1528         JadwalTayang(String jamMulai) {
1529             this.jamMulai = jamMulai.trim();
1530         }
1531
1532         JadwalTayang copy() {
1533             JadwalTayang j = new JadwalTayang(jamMulai);
1534             for (int i = 0; i < ROWS; i++) {
1535                 j.kursi[i] = Arrays.copyOf(kursi[i], COLS);
1536             }
1537             return j;
1538         }
1539
1540         LocalTime getJamMulai() {
1541             return LocalTime.parse(jamMulai, TIME_FMT);
1542         }
1543
1544         LocalTime getJamSelesai(int durasiMenit) {
1545             return getJamMulai().plusMinutes(durasiMenit);
1546         }
1547
1548         boolean isOccupied(String seatCode) {
1549             int[] rc = parseSeat(seatCode);
1550             return rc != null && kursi[rc[0]][rc[1]];
1551         }
1552
1553         void occupy(String seatCode) {
1554             int[] rc = parseSeat(seatCode);
1555             if (rc != null) kursi[rc[0]][rc[1]] = true;
1556         }
1557

```

```

1558         int countOccupied() {
1559             int count = 0;
1560             for (int r = 0; r < ROWS; r++) {
1561                 for (int c = 0; c < COLS; c++) {
1562                     if (kursi[r][c]) count++;
1563                 }
1564             }
1565             return count;
1566         }
1567
1568         String serialize() {
1569             StringBuilder bits = new StringBuilder();
1570             for (int r = 0; r < ROWS; r++) {
1571                 for (int c = 0; c < COLS; c++) {
1572                     bits.append(kursi[r][c] ? '1' : '0');
1573                 }
1574             }
1575             return jamMulai + "#" + bits;
1576         }
1577

```

```

1578         static JadwalTayang deserialize(String token) {
1579             if (token == null || token.isBlank()) return null;
1580             String[] p = token.split("#", -1);
1581             if (p.length < 2) return null;
1582             String jam = p[0].trim();
1583             if (!jam.matches("^\\d{2}:\\d{2}$")) return null;
1584             JadwalTayang j = new JadwalTayang(jam);
1585             String bits = p[1].trim();
1586             int idx = 0;
1587             for (int r = 0; r < ROWS; r++) {
1588                 for (int c = 0; c < COLS; c++) {
1589                     if (idx < bits.length()) {
1590                         j.kursi[r][c] = bits.charAt(idx) == '1';
1591                     }
1592                     idx++;
1593                 }
1594             }
1595             return j;
1596         }
1597     }
1598 }

```

SistemBioskop.Java - Bagian 20: class Tansaksis + parseSeat () + penutup kelas

```

1599 private static class Transaksi {
1600     String idtransaksi;
1601     String tanggal;
1602     String kodeFilm;
1603     String judulFilm;
1604     String genreFilm;
1605     int studio;
1606     String jamMulai;
1607     String jamSelesai;
1608     int durasiFilm;
1609     double hargaPerTiket;
1610     int jumlahTiket;
1611     List<String> kursiDipilih;
1612     double totalBayar;
1613     double uangBayar;
1614     double kembalian;
1615 }

```

```

1616 Transaksi(String idtransaksi, String tanggal, String kodeFilm, String judulFilm, String genreFilm, int studio,
1617           String jamMulai, String jamSelesai, int durasiFilm, double hargaPerTiket, int jumlahTiket,
1618           List<String> kursiDipilih, double totalBayar, double uangBayar, double kembalian) {
1619     this.idtransaksi = idtransaksi;
1620     this.tanggal = tanggal;
1621     this.kodeFilm = kodeFilm;
1622     this.judulFilm = judulFilm;
1623     this.genreFilm = genreFilm;
1624     this.studio = studio;
1625     this.jamMulai = jamMulai;
1626     this.jamSelesai = jamSelesai;
1627     this.durasiFilm = durasiFilm;
1628     this.hargaPerTiket = hargaPerTiket;
1629     this.jumlahTiket = jumlahTiket;
1630     this.kursiDipilih = kursiDipilih;
1631     this.totalBayar = totalBayar;
1632     this.uangBayar = uangBayar;
1633     this.kembalian = kembalian;
1634 }
1635
1636 private static int[] parseSeat(String seatCode) {
1637     if (seatCode == null) return null;
1638     seatCode = seatCode.trim().toUpperCase(Locale.ROOT);
1639     if (InvalidSeatCode(seatCode)) return null;
1640     int row = seatCode.charAt(0) - 'A';
1641     int col = Integer.parseInt(seatCode.substring(1)) - 1;
1642     return new int[]{row, col};
1643 }
1644 }
1645 }

```

3.3 Ringkasan Metode Implementasi

Tabel berikut merangkum seluruh metode utama yang diimplementasikan beserta fungsi dan kategorinya dalam program.

Nama Metode	Deskripsi Fungsi	Kategori
<code>main()</code>	Entry point: load data, seed, rekonsiliasi, loop menu utama	Entry Point
<code>menuKelolaFilm()</code>	Sub-menu pengelolaan film (CRUD + jadwal)	Menu
<code>tambahFilm()</code>	CRUD Create: validasi regex kode, input semua atribut, simpan ke CSV	CRUD
<code>tampilkanDaftarFilm()</code>	CRUD Read: render tabel film dengan jadwal dan sisa kursi	CRUD
<code>editFilm()</code>	CRUD Update: pola temporary object, konfirmasi sebelum simpan	CRUD
<code>editJadwalFilm()</code>	Kelola jadwal (tambah/ubah/hapus) dengan validasi kursi terisi	CRUD
<code>hapusFilm()</code>	CRUD Delete: soft delete — ubah aktif=false tanpa hapus baris	CRUD
<code>containsIgnoreCaseManual()</code>	Linear Search substring manual case-insensitive — $O(n \times m)$	Searching
<code>cariFilmByKeyword()</code>	Cari film aktif by keyword (kode / judul / genre)	Searching
<code>cariFilmByKode()</code>	Sequential Search film by kode — $O(n)$	Searching
<code>filterFilmByGenre()</code>	Filter kategori genre dengan LinkedHashMap + Linear Search	Searching

<code>cariTransaksiByIdInternal()</code>	Sequential Search transaksi by ID — $O(n)$	Searching
<code>urutkanFilmManual()</code>	Selection Sort manual — $O(n^2)$ — 3 mode	Sorting
<code>perluTukarUrutan()</code>	Komparator manual untuk 3 mode sort (kode/judul/tiket)	Sorting
<code>bandingkanTextManual()</code>	Perbandingan string karakter per karakter tanpa <code>compareTo()</code>	Sorting
<code>transaksiTiketBaru()</code>	State machine 7 tahap: pilih film → jadwal → kursi → bayar	Transaksi
<code>tampilkanDenahKursi()</code>	Render visualisasi matriks kursi $[X]/[]$ real-time di konsol	Transaksi
<code>menuLaporan()</code>	Statistik real-time: total transaksi, tiket, pendapatan, film terlaris	Laporan
<code>printInvoice()</code>	Cetak nota transaksi detail ke konsol	Laporan
<code>saveFilms()</code> / <code>saveTransactions()</code>	Persistensi data ke <code>films.csv</code> dan <code>transactions.csv</code>	File I/O
<code>loadFilms()</code> / <code>loadTransactions()</code>	Muat data dari CSV ke <code>ArrayList</code> saat startup	File I/O
<code>reconcileSeatsFromTransactions()</code>	Sinkronisasi status kursi dari riwayat transaksi saat startup	File I/O
<code>rebuildTicketStats()</code>	Hitung ulang counter tiketTerjual dari file transaksi	File I/O
<code>normalizeNumberInput()</code>	Normalisasi multi-format input uang (Rp50.000 / 50,000 / 50000)	Validasi
<code>JadwalTayang.serialize()</code>	Serialisasi matriks <code>boolean[5][10]</code> ke string biner 50-bit	Model Data

<code>JadwalTayang.deserialize()</code>	Deserialisasi string biner kembali ke matriks boolean[5][10]	Model Data
<code>Film.copy()</code>	Deep copy objek Film beserta jadwal (untuk pola temporary object)	Model Data
<code>parseSeat()</code>	Konversi kode kursi 'A1'-'E10' ke indeks array [row][col]	Model Data

3.4 Kesimpulan Impelementasi

Implementasi Sistem Pemesanan Tiket Bioskop ini berhasil memenuhi seluruh persyaratan fungsional yang ditetapkan dalam soal UAS. Seluruh algoritma pencarian dan pengurutan diimplementasikan secara manual tanpa menggunakan library eksternal. Fitur tambahan berupa persistensi CSV, rekonsiliasi data otomatis, denah kursi interaktif, dan laporan statistik real-time turut diimplementasikan sebagai nilai tambah pada kategori Fitur Tambahan (bonus).

Struktur program yang modular — dengan pemisahan jelas antara model data, logika bisnis, algoritma, dan antarmuka — memudahkan pemeliharaan kode dan memenuhi kriteria penilaian Code Readability. Pola-pola desain yang diterapkan seperti temporary object untuk edit aman, state machine untuk wizard transaksi, dan snapshot memory untuk transaksi menunjukkan pemahaman mendalam terhadap konsep pemrograman berorientasi objek dan pengelolaan state aplikasi.

BAB IV

ANALISIS KOMPLEKSITAS

Pada sistem pemesanan tiket bioskop ini, analisis kompleksitas dilakukan berdasarkan proses utama yang terdapat pada kode program. Setiap fungsi dikelompokkan agar pembahasan lebih terstruktur dan tidak terlalu padat dalam satu tabel. Analisis dilakukan menggunakan notasi Big-O untuk mengetahui tingkat efisiensi algoritma berdasarkan waktu eksekusi dan kebutuhan memori yang digunakan program.

4.1 Tabel Analisis Kompleksitas

4.1.1 Analisis Kompleksitas Searching

Fungsi	Algoritma/Proses yang Digunakan	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
cariFilmByKode()	Linear search	$O(n)$	$O(1)$	Mencari data film satu per satu sampai kode film ditemukan.
cariFilmByKodeAktif()	Linear Search + pengecekan status	$O(n)$	$O(1)$	Kompleksitas mengikuti pencarian film, lalu ditambah validasi status aktif.
cariTransaksiByIdInternal() ()	Linear Search	$O(t)$	$O(1)$	Mencari transaksi berdasarkan ID dengan

				memeriksa seluruh data transaksi.
cariFilmByKeyword()	Pencocokan string berulang	$O(n \cdot m)$	$O(h)$	Memeriksa keyword pada seluruh film aktif, bergantung pada jumlah film dan panjang teks.
cariFilmByGenre()	Pencocokan string berulang	$O(n \cdot g)$	$O(h)$	Mencari film berdasarkan genre melalui pemeriksaan pada seluruh data film aktif.

4.1.2 Analisis Kompleksitas Sorting dan Tampilan Data

Fungsi	Algoritma/Proses yang Digunakan	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
urutkanFilmManual()	Selection Sort	$O(n^2)$	$O(1)$	Menggunakan dua perulangan bersarang untuk

				menentukan urutan film.
tampilkanFilmTerurut())	Penyalinan data + Selection sorting	$O(n^2)$	$O(n)$	Data disalin terlebih dahulu, lalu diurutkan menggunakan proses manual.
hitungKursiTersedia()	Traversal matriks	$O(r \times c)$	$O(1)$	Menampilkan seluruh kursi dengan menelusuri baris dan kolom satu per satu.
tampilkanDenahKursi())	Traversal matriks	$O(r \times c)$	$O(1)$	Menghitung jumlah kursi berdasarkan status pada matriks kursi.

4.1.3 Analisis Kompleksitas Transaksi

Fungsi	Algoritma/Proses yang Digunakan	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
transaksiTiketBaru()	Pencarian data + validasi kursi	$O(n + s + k \cdot r \cdot c)$	$O(k)$	Melibatkan pencarian film, pemilihan jadwal,

				dan validasi kursi yang dipilih.
generateNextTransactionId()	Pencarian nilai maksimum	$O(t)$	$O(1)$	Menelusuri seluruh transaksi untuk menentukan nomor ID berikutnya.
rebuildTicketStats()	Reset data + rekap transaksi	$O(n + t)$	$O(1)$	Statistik tiket dihitung ulang dari daftar film dan transaksi yang tersimpan.
reconcileSeatsFromTransactions()	Sinkronisasi data kursi	$O(t \cdot k)$	$O(1)$	Setiap transaksi dibaca, lalu kursi di dalam transaksi ditandai kembali sebagai terisi.

4.1.4 Analisis Kompleksitas Load dan Save CSV

Fungsi	Algoritma/Proses yang Digunakan	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
saveFilms()	Serialisasi data ke CSV	$O(n \cdot s \cdot r \cdot c)$	$O(1)$	Seluruh film, jadwal, dan status kursi ditulis ke file CSV.
saveTransactions()	Penulisan berurutan ke CSV	$O(t)$	$O(1)$	Setiap transaksi disimpan satu per satu ke file.
loadFilms()	Deserialisasi data dari CSV	$O(n \cdot s \cdot r \cdot c)$	$O(n)$	Data film dan jadwal dibaca kembali lalu dibentuk ulang ke dalam struktur program.
loadTransactions()	Pembacaan berurutan dari CSV	$O(t)$	$O(t)$	Seluruh transaksi dimuat ke memori dari file CSV.

Keterangan notasi:

n = jumlah film

t = jumlah transaksi

s = jumlah jadwal per film

k = jumlah kursi yang dipilih dalam satu transaksi

m = panjang teks keyword pencarian

g = panjang teks genre yang dibandingkan

$r \times c$ = ukuran matriks kursi

h = kebutuhan memori tambahan pada proses manipulasi teks

Pada implementasi program ini, ukuran kursi ditetapkan **5 baris × 10 kolom**, sehingga $r = 5$ dan $c = 10$. Karena ukuran matriks kursi bersifat tetap, proses yang berkaitan dengan denah kursi memiliki beban komputasi yang relatif stabil dan tidak bertambah besar secara signifikan.

4.2 Penjelasan Analisis

Dari tabel di atas, terlihat bahwa bagian yang paling mahal secara komputasi adalah **sorting manual** dan **proses baca/tulis data film yang melibatkan jadwal serta matriks kursi**. Hal ini wajar karena program melakukan pengolahan data dengan pendekatan manual, tanpa memanfaatkan library sorting bawaan, dan menyimpan status kursi pada matriks boolean di setiap jadwal.

Fungsi pencarian film dan transaksi masih tergolong efisien untuk data skala kecil hingga menengah karena hanya menggunakan **linear search**. Namun, ketika jumlah film dan transaksi bertambah besar, pencarian dan sorting akan menjadi lebih lambat karena seluruh data harus dipindai berulang. Sebaliknya, operasi kursi relatif aman dari sisi performa karena jumlah kursi per jadwal tetap, yaitu 50 kursi.

Dari sisi ruang, penggunaan `ArrayList` untuk film dan transaksi membuat kebutuhan memori bertambah secara linear sesuai jumlah data. Struktur ini cocok untuk aplikasi console seperti pada project ini, karena implementasinya sederhana dan mudah disimpan ke CSV.

4.3 Kesimpulan Analisis Kompleksitas

Berdasarkan analisis kompleksitas, dapat disimpulkan bahwa sistem pemesanan tiket bioskop ini sudah cukup baik untuk penggunaan skala tugas atau data kecil-menengah. Operasi utama yang perlu diperhatikan adalah pencarian linear, sorting selection sort, serta proses simpan/muat data CSV. Meski demikian, penggunaan matriks kursi tetap membuat pengelolaan seat menjadi sederhana dan konsisten.

BAB V

PENGUJIAN

Bab ini bertujuan untuk memastikan seluruh fitur pada sistem pemesanan tiket bioskop dapat berjalan sesuai kebutuhan yang telah dirancang. Pengujian dilakukan berdasarkan masukan dan keluaran sistem tanpa melihat implementasi kode secara langsung.

Pengujian difokuskan pada fitur utama sistem serta validasi kondisi tidak valid untuk memastikan sistem dapat menangani kesalahan input dengan baik.

5.1 Skenario Pengujian Sistem

No	Skenario Pengujian	Input	Hasil yang Diharapkan	Hasil Uji Coba	Status
1	Mengisi tanggal tidak sesuai formatnya	21/05/26	Sistem menolak input	Muncul pesan kesalahan	Berhasil
2	Menambahkan kode film yang sudah digunakan	F001	Sistem menolak penambahan	Muncul pesan kesalahan	Berhasil
3	Memilih kursi yang sudah terisi	C1	Sistem menolak pemilihan	Muncul pesan kesalahan	Berhasil
4	Memasukkan kode kursi tidak valid	kursi1	Sistem menolak input	Muncul pesan kesalahan	Berhasil
5	Memasukkan jumlah tiket melebihi kapasitas	100	Sistem menolak input	Muncul pesan kesalahan	Berhasil
6	Melakukan pembayaran kurang dari total pembayaran	30000	Sistem menolak pembayaran	Muncul pesan kesalahan	Berhasil
7	Mencari film berdasarkan genre	Drama	Sistem menampilkan daftar film	Film dengan genre dipilih	Berhasil

			dengan genre yang sesuai	berhasil muncul	
8	Mengurutkan film berdasarkan jumlah tiket penjualan	-	Sistem menampilkan daftar film berdasarkan jumlah tiket terjual	Film berhasil ditampilkan sesuai urutan jumlah tiket penjualan	Berhasil

5.2 Dokumentasi Hasil Pengujian

1. Tanggal Tidak Sesuai Format

Tujuan: memastikan sistem menolak input tanggal yang tidak sesuai format.

Screenshot:

```
=====
                        TRANSAKSI TIKET BARU
=====
Masukkan tanggal transaksi (dd/MM/yyyy) [C batal]: 21/05/26
[X] Format tanggal harus dd/MM/yyyy, contoh 27/01/2026.
```

2. Kode Film Duplikat

Tujuan: memastikan sistem menolak kode film yang sudah digunakan.

Screenshot:

```
=====
                        TAMBAH FILM
=====
Kode film (format F001) [C batal]: f001
[X] Kode film sudah dipakai.
```

3. Memilih Kursi yang Sudah Terisi

Tujuan: memastikan sistem menolak pemilihan kursi yang telah digunakan.

Screenshot:

```
----- LAYAR -----
      1  2  3  4  5  6  7  8  9 10
A  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
B  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
C  [X] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
D  [ ] [ ] [ ] [X] [X] [ ] [ ] [ ] [ ] [ ]
E  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
Keterangan: [ ] Tersedia, [X] Terisi / Dipilih
Kursi sementara : -
Pilih kursi ke-1 [B kembali, C batal]: c1
[X] Kursi sudah terisi atau sudah dipilih.
```

4. Kode Kursi Tidak Valid

Tujuan: memastikan sistem melakukan validasi kode kursi.

Screenshot:

```
----- LAYAR -----
      1  2  3  4  5  6  7  8  9 10
A  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
B  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
C  [X] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
D  [ ] [ ] [ ] [X] [X] [ ] [ ] [ ] [ ] [ ]
E  [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
Keterangan: [ ] Tersedia, [X] Terisi / Dipilih
Kursi sementara : -
Pilih kursi ke-1 [B kembali, C batal]: kursi1
[X] Format kursi salah. Contoh: A1, C10.
```

5. Jumlah Tiket Melebihi Kapasitas

Tujuan: memastikan sistem menolak jumlah tiket yang melebihi kapasitas kursi tersedia.

Screenshot:

```
Masukkan jumlah tiket [B kembali, C batal]: 100
[!] Kursi tersedia hanya 47.
```

6. Pembayaran Kurang dari Total Pembayaran

Tujuan: memastikan sistem menolak pembayaran yang lebih kecil dari total transaksi.

Screenshot:

```
----- RINGKASAN TRANSAKSI -----
Tanggal      : 27/06/2026
Film         : Tunggu Aku Sukses Nanti
Genre        : Drama
Studio       : 5
Jam mulai    : 11:30
Jam selesai  : 13:30
Jumlah tiket : 1
Kursi        : A1
Harga/tiket  : Rp40.000
Total        : Rp40.000

Tekan ENTER untuk lanjut ke pembayaran, atau [B kembali, C batal]:
Masukkan uang pembayaran [B kembali, C batal]: Rp30000
[X] Uang tidak mencukupi. Minimal Rp40.000
```

7. Mencari Film Berdasarkan Genre

Tujuan: memastikan sistem dapat menampilkan film berdasarkan genre yang dipilih.

Screenshot:

```
----- GENRE TERSEDIA -----
1. Drama          2 film
2. Comedy          2 film
3. Adventure       1 film
4. Thriller        1 film
5. Sci-Fi          1 film
6. Romance         1 film
7. Animation       1 film
8. Horror          1 film
Masukkan genre [C batal]: drama
----- HASIL PENCARIAN GENRE -----
-----DAFTAR FILM-----
No  Kode  Judul                Genre    Studio  Durasi  Status
-----
1   F001   Tunggu Aku Sukses Nanti  Drama    5       120m   Aktif
    Harga : Rp40.000 | Terjual: 5 tiket
    Jadwal : 11:30 (47 kursi), 14:00 (48 kursi)

2   F003   Ayah, Ini Arahnya Kemana? Drama    1       103m   Aktif
    Harga : Rp50.000 | Terjual: 5 tiket
    Jadwal : 19:00 (47 kursi), 21:00 (48 kursi)
```

8. Mengurutkan Film Berdasarkan Jumlah Tiket Penjualan

Tujuan: memastikan sistem dapat mengurutkan data film berdasarkan jumlah tiket terjual.

Screenshot:

URUT TIKET FILM						
-DAFTAR FILM-						
No	Kode	Judul	Genre	Studio	Durasi	Status
1	F010	Arsip Rumah Tua	Horror	5	99m	Aktif
		Harga : Rp48.000 Terjual: 10 tiket				
		Jadwal : 17:45 (45 kursi), 22:00 (45 kursi)				
2	F004	Laut di Atas Awan	Adventure	3	112m	Aktif
		Harga : Rp55.000 Terjual: 10 tiket				
		Jadwal : 10:15 (47 kursi), 13:00 (49 kursi), 18:30 (44 kursi)				
3	F009	Negeri Lupa Peta	Animation	2	88m	Aktif
		Harga : Rp38.000 Terjual: 9 tiket				
		Jadwal : 09:30 (48 kursi), 12:00 (46 kursi), 16:00 (47 kursi)				
4	F008	Hujan di Stasiun	Romance	8	101m	Aktif
		Harga : Rp47.000 Terjual: 7 tiket				
		Jadwal : 13:20 (45 kursi), 18:10 (48 kursi)				
5	F006	Orbit Terakhir	Sci-Fi	6	130m	Aktif
		Harga : Rp60.000 Terjual: 7 tiket				
		Jadwal : 11:00 (47 kursi), 14:45 (49 kursi), 19:15 (47 kursi)				
6	F005	Kode Merah Senja	Thriller	4	105m	Aktif
		Harga : Rp50.000 Terjual: 6 tiket				
		Jadwal : 12:30 (48 kursi), 16:15 (49 kursi), 20:30 (47 kursi)				
7	F002	Senin Harga Naik	Comedy	2	98m	Aktif
		Harga : Rp45.000 Terjual: 6 tiket				
		Jadwal : 15:00 (46 kursi), 17:00 (48 kursi)				
8	F003	Ayah, Ini Arahnya Kemana?	Drama	1	103m	Aktif
		Harga : Rp50.000 Terjual: 5 tiket				
		Jadwal : 19:00 (47 kursi), 21:00 (48 kursi)				

5.3 Analisis Hasil Pengujian

Berdasarkan delapan skenario pengujian yang dilakukan, sistem berhasil menjalankan fungsi utama dan validasi dengan baik. Selain dapat melakukan proses normal seperti penambahan film, sistem juga mampu menangani berbagai kondisi kesalahan seperti data duplikat, kursi tidak valid, kapasitas tiket berlebih, serta pembayaran yang tidak mencukupi.

BAB VI

KESIMPULAN

Sistem Manajemen Bioskop berbasis Java yang kami buat ini berhasil dirancang dan diimplementasikan sebagai solusi digital terintegrasi untuk mengotomatisasi seluruh alur kerja kasir dan administrasi operasional bioskop. Melalui struktur data pemrograman berorientasi objek dan sistem penyimpanan permanen berbasis berkas teks CSV, aplikasi ini mampu mengelola data master film secara dinamis, menyimpan riwayat transaksi secara aman, serta meminimalkan kesalahan manusia dalam pemilihan nomor kursi lewat denah studio digital. Semua fitur fungsional utama, seperti manajemen data dengan metode penghapusan logis, panduan transaksi bertahap, serta algoritma pencarian dan pengurutan manual, dapat beroperasi secara mandiri tanpa bergantung pada pustaka eksternal tambahan.

Berdasarkan analisis performa, sistem ini dinilai sangat baik dan stabil untuk menangani pengelolaan data skala kecil hingga menengah. Penerapan fungsi rekonsiliasi data otomatis saat aplikasi dijalankan terbukti mampu menjaga konsistensi status keterisian kursi studio agar selalu akurat dan sinkron dengan data transaksi *real* di lapangan. Selain itu, hasil pengujian menyeluruh menunjukkan bahwa sistem tidak hanya berhasil menjalankan proses operasional normal, tetapi juga memiliki tingkat validasi yang ketat dan responsif dalam menangani berbagai kondisi kesalahan input, seperti format tanggal yang salah, data duplikat, pemilihan kursi yang sudah terisi, hingga kekurangan nominal pembayaran. Secara keseluruhan, pengembangan program ini telah memenuhi semua persyaratan yang ditetapkan serta berhasil menyajikan laporan ringkasan pendapatan dan statistik film terlaris secara langsung guna mendukung efisiensi manajemen bisnis bioskop.